```
!pip install pandas-ta yfinance
```

```
Collecting pandas-ta
  Downloading pandas_ta-0.4.71b0-py3-none-any.whl.metadata (2.3 kB)
Requirement already satisfied: yfinance in /usr/local/lib/python3.12/dist-packages (0.2.66)
Collecting numba==0.61.2 (from pandas-ta)
  Downloading numba-0.61.2-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.whl.metadata (2.8 kB)
Collecting numpy>=2.2.6 (from pandas-ta)
  Downloading numpy-2.4.4-cp312-cp312-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl.metadata (6.6 kB)
Collecting pandas>=2.3.2 (from pandas-ta)
  Downloading pandas-3.0.2-cp312-cp312-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl.metadata (79 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 79.5/79.5 kB 2.2 MB/s eta 0:00:00
Requirement already satisfied: tqdm>=4.67.1 in /usr/local/lib/python3.12/dist-packages (from pandas-ta) (4.67.3)
Collecting llvmlite<0.45,>=0.44.0dev0 (from numba==0.61.2->pandas-ta)
  Downloading llvmlite-0.44.0-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (5.0 kB)
Collecting numpy>=2.2.6 (from pandas-ta)
  Downloading numpy-2.2.6-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (62 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 62.0/62.0 kB 2.4 MB/s eta 0:00:00
Requirement already satisfied: requests>=2.31 in /usr/local/lib/python3.12/dist-packages (from yfinance) (2.32.4)
Requirement already satisfied: multitasking>=0.0.7 in /usr/local/lib/python3.12/dist-packages (from yfinance) (0.0.12)
Requirement already satisfied: platformdirs>=2.0.0 in /usr/local/lib/python3.12/dist-packages (from yfinance) (4.9.4)
Requirement already satisfied: pytz>=2022.5 in /usr/local/lib/python3.12/dist-packages (from yfinance) (2025.2)
Requirement already satisfied: frozendict>=2.3.4 in /usr/local/lib/python3.12/dist-packages (from yfinance) (2.4.7)
Requirement already satisfied: peewee>=3.16.2 in /usr/local/lib/python3.12/dist-packages (from yfinance) (4.0.3)
Requirement already satisfied: beautifulsoup4>=4.11.1 in /usr/local/lib/python3.12/dist-packages (from yfinance) (4.13.5)
Requirement already satisfied: curl_cffi>=0.7 in /usr/local/lib/python3.12/dist-packages (from yfinance) (0.14.0)
Requirement already satisfied: protobuf>=3.19.0 in /usr/local/lib/python3.12/dist-packages (from yfinance) (5.29.6)
Requirement already satisfied: websockets>=13.0 in /usr/local/lib/python3.12/dist-packages (from yfinance) (15.0.1)
Requirement already satisfied: soupsieve>1.2 in /usr/local/lib/python3.12/dist-packages (from beautifulsoup4>=4.11.1->yfin
Requirement already satisfied: typing-extensions>=4.0.0 in /usr/local/lib/python3.12/dist-packages (from beautifulsoup4>=4
Requirement already satisfied: cffi>=1.12.0 in /usr/local/lib/python3.12/dist-packages (from curl_cffi>=0.7->yfinance) (2.
Requirement already satisfied: certifi>=2024.2.2 in /usr/local/lib/python3.12/dist-packages (from curl_cffi>=0.7->yfinance
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas>=2.3.2->pand
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests>=2.31->y
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests>=2.31->yfinance) (3.
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests>=2.31->yfinanc
Requirement already satisfied: pycparser in /usr/local/lib/python3.12/dist-packages (from cffi>=1.12.0->curl_cffi>=0.7->yf
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pandas>=2
Downloading pandas_ta-0.4.71b0-py3-none-any.whl (240 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 240.3/240.3 kB 9.5 MB/s eta 0:00:00
Downloading numba-0.61.2-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.whl (3.9 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 3.9/3.9 MB 56.7 MB/s eta 0:00:00
Downloading numpy-2.2.6-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (16.5 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 16.5/16.5 MB 73.9 MB/s eta 0:00:00
Downloading pandas-3.0.2-cp312-cp312-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl (10.9 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 10.9/10.9 MB 94.4 MB/s eta 0:00:00
Downloading llvmlite-0.44.0-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (42.4 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 42.4/42.4 MB 14.1 MB/s eta 0:00:00
Installing collected packages: numpy, llvmlite, pandas, numba, pandas-ta
  Attempting uninstall: numpy
    Found existing installation: numpy 2.0.2
    Uninstalling numpy-2.0.2:
      Successfully uninstalled numpy-2.0.2
  Attempting uninstall: llvmlite
    Found existing installation: llvmlite 0.43.0
    Uninstalling llvmlite-0.43.0:
      Successfully uninstalled llvmlite-0.43.0
  Attempting uninstall: pandas
    Found existing installation: pandas 2.2.2
    Uninstalling pandas-2.2.2:
      Successfully uninstalled pandas-2.2.2
  Attempting uninstall: numba
    Found existing installation: numba 0.60.0
    Uninstalling numba-0.60.0:
      Successfully uninstalled numba-0.60.0
ERROR: pip's dependency resolver does not currently take into account all the packages that are installed. This behaviour
google-colab 1.0.0 requires pandas==2.2.2, but you have pandas 3.0.2 which is incompatible.
db-dtypes 1.5.1 requires pandas<3.0.0,>=1.5.3, but you have pandas 3.0.2 which is incompatible.
tensorflow 2.19.0 requires numpy<2.2.0,>=1.26.0, but you have numpy 2.2.6 which is incompatible.
bqplot 0.12.45 requires pandas<3.0.0,>=1.0.0, but you have pandas 3.0.2 which is incompatible.
gradio 5.50.0 requires pandas<3.0,>=1.0, but you have pandas 3.0.2 which is incompatible.
Successfully installed llvmlite-0.44.0 numba-0.61.2 numpy-2.2.6 pandas-3.0.2 pandas-ta-0.4.71b0
```

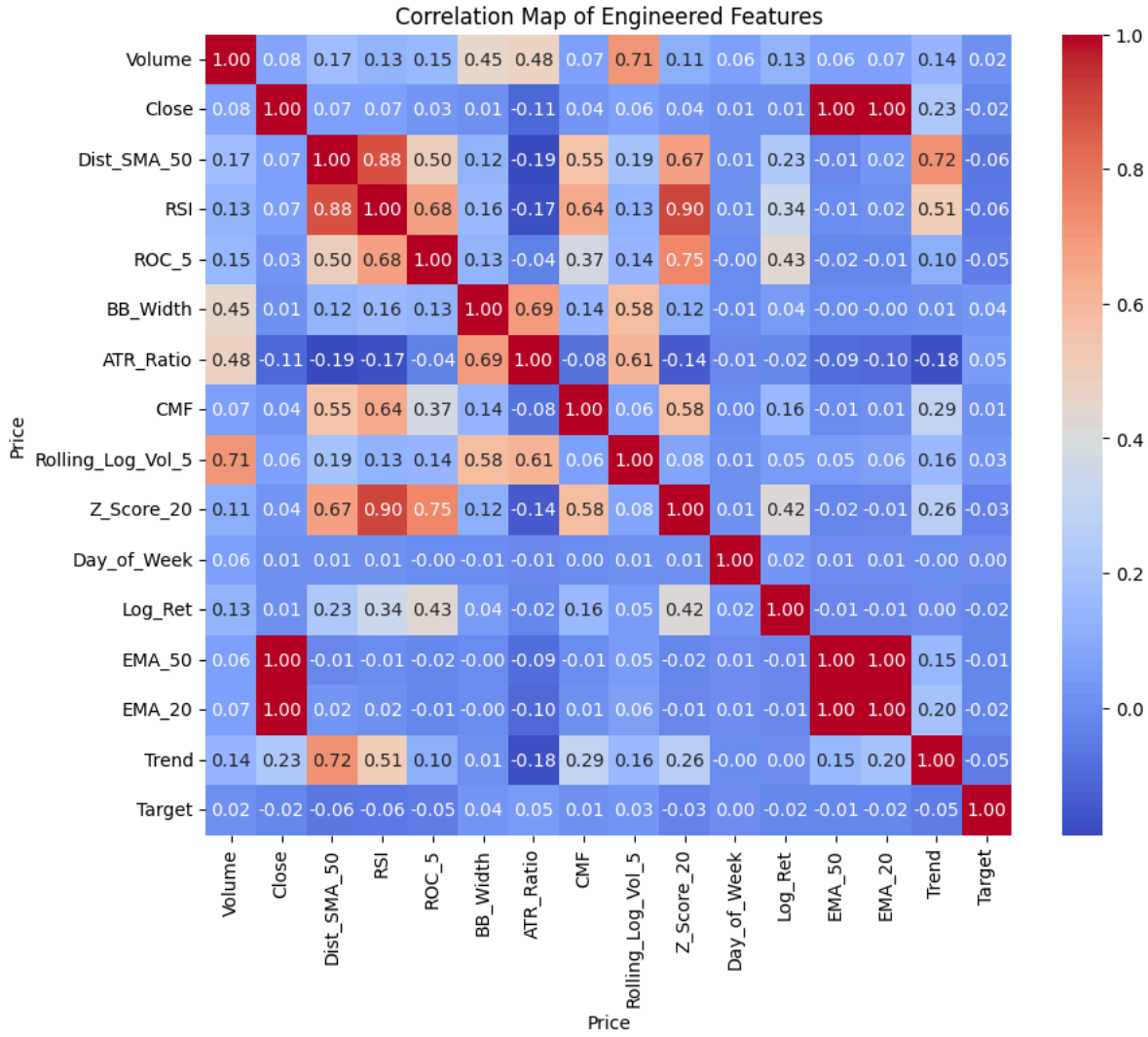
```
import yfinance as yf

# 1. Load Data
df = yf.download("RELIANCE.NS", start="2011-01-01", end="2024-01-01")
```

```
/tmp/ipykernel_1361/612459453.py:4: FutureWarning: YF.download() has changed argument auto_adjust default to True
  df = yf.download("RELIANCE.NS", start="2011-01-01", end="2024-01-01")
/usr/local/lib/python3.12/dist-packages/yfinance/scrapers/history.py:204: Pandas4Warning: Timestamp.utcnow is deprecated and
  dt_now = pd.Timestamp.utcnow()
[**********100%**********]  1 of 1 completed
```

```
import pandas as pd
import numpy as np
import pandas_ta as ta
import matplotlib.pyplot as plt
import seaborn as sns
```

```
# --- FIX: Flatten MultiIndex columns if they exist ---
# The yfinance.download function can return a DataFrame with MultiIndex columns,
# which causes issues when assigning new single-level named columns.
# This converts columns like ('Close', 'RELIANCE.NS') to 'Close'.
if isinstance(df.columns, pd.MultiIndex):
    df.columns = df.columns.get_level_values(0)

# --- FEATURE ENGINEERING ---

# A. Moving Average Convergence / Basics
# Distance from SMA (50-day) - Measures mean reversion
df['SMA_50'] = df['Close'].rolling(window=50).mean()
df['Dist_SMA_50'] = (df['Close'] / df['SMA_50']) - 1

# B. Momentum & Trend
# RSI (14-day)
df['RSI'] = ta.rsi(df['Close'], length=14)

# ROC (Rate of Change - 5-day)
# Captures the percentage change over the prediction horizon
df['ROC_5'] = ta.roc(df['Close'], length=5)

# C. Volatility & Risk
# Bollinger Band Width
# (Upper - Lower) / Mid. A low value indicates a "Squeeze"
bbands = ta.bbands(df['Close'], length=20, std=2)
df['BB_Width'] = (bbands['BBU_20_2.0_2.0'] - bbands['BBL_20_2.0_2.0']) / bbands['BBM_20_2.0_2.0']

# ATR Ratio (Normalized Volatility)
# Dividing by Close makes it comparable across 13 years of price action
df['ATR'] = ta.atr(df['High'], df['Low'], df['Close'], length=14)
df['ATR_Ratio'] = df['ATR'] / df['Close']

# D. Volume & Flow
# Chaikin Money Flow (CMF) - Measures accumulation vs distribution
df['CMF'] = ta.cmf(df['High'], df['Low'], df['Close'], df['Volume'], length=20)

# Rolling Log Volume (5-day)
# We take the log to normalize spikes, then calculate a rolling sum/mean
df['Log_Vol'] = np.log(df['Volume'])
df['Rolling_Log_Vol_5'] = df['Log_Vol'].rolling(window=5).mean()

# E. Statistical Stationary Features
# Z-Score (20-day) - How many std devs is current price from the mean?
df['Z_Score_20'] = (df['Close'] - df['Close'].rolling(20).mean()) / df['Close'].rolling(20).std()

# F. Time-Based Features
# Day of the Week (Monday=0, Sunday=6)
df['Day_of_Week'] = df.index.dayofweek

df['EMA_20'] = ta.ema(df['Close'], length=20)
df['EMA_50'] = ta.ema(df['Close'], length=50)
df['Trend'] = df['EMA_20'] - df['EMA_50']

df['ADX'] = ta.adx(df['High'], df['Low'], df['Close'])['ADX_14']
df['Momentum'] = ta.mom(df['Close'], length=10)

# --- Create Log Returns and Target Variable ---
df['Log_Ret'] = np.log(df['Close']).diff()
df['Target'] = (df['Close'].shift(-5) > df['Close']).astype(int)

# --- CLEANUP ---
# Drop rows with NaNs created by rolling windows (e.g., SMA_50 needs 50 days) and new features
df.dropna(inplace=True)

# Select features for correlation map
features_to_correlate = [
    'Volume',
    'Close',
    'Dist_SMA_50',
    'RSI',
    'ROC_5',
    'BB_Width',
    'ATR_Ratio',
    'CMF',
```

```
    'Rolling_Log_Vol_5',
    'Z_Score_20',
    'Day_of_Week',
    'Log_Ret',
    'EMA_50',
    'EMA_20',
    'Trend',
    'Target'
]

# Calculate the correlation matrix
correlation_matrix = df[features_to_correlate].corr()

# Create a heatmap
plt.figure(figsize=(10, 8))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', fmt=".2f")
plt.title('Correlation Map of Engineered Features')
plt.show()
```




```
!pip uninstall numpy scikit-learn
!pip install numpy==1.26.4
```

```
Found existing installation: numpy 1.26.4
Uninstalling numpy-1.26.4:
  Would remove:
    /usr/local/bin/f2py
    /usr/local/lib/python3.12/dist-packages/numpy-1.26.4.dist-info/*
    /usr/local/lib/python3.12/dist-packages/numpy.libs/libgfortran-040039e1.so.5.0.0
    /usr/local/lib/python3.12/dist-packages/numpy.libs/libopenblas64_p-r0-0cf96a72.3.23.dev.so
    /usr/local/lib/python3.12/dist-packages/numpy.libs/libquadmath-96973f99.so.0.0.0
    /usr/local/lib/python3.12/dist-packages/numpy/*
Proceed (Y/n)? y
  Successfully uninstalled numpy-1.26.4
WARNING: Skipping scikit-learn as it is not installed.
Collecting numpy==1.26.4
  Using cached numpy-1.26.4-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (61 kB)
Using cached numpy-1.26.4-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (18.0 MB)
Installing collected packages: numpy
ERROR: pip's dependency resolver does not currently take into account all the packages that are installed. This behaviour
pynndescent 0.6.0 requires scikit-learn>=0.18, which is not installed.
yellowbrick 1.5 requires scikit-learn>=1.0.0, which is not installed.
spreg 1.9.0 requires scikit-learn>=0.22, which is not installed.
tsfresh 0.21.1 requires scikit-learn>=0.22.0, which is not installed.
sklearn-pandas 2.2.0 requires scikit-learn>=0.23.0, which is not installed.
segregation 2.5.4 requires scikit-learn>=0.21.3, which is not installed.
shap 0.51.0 requires scikit-learn, which is not installed.
sentence-transformers 5.3.0 requires scikit-learn, which is not installed.
esda 2.9.0 requires scikit-learn>=1.4, which is not installed.
umap-learn 0.5.11 requires scikit-learn>=1.6, which is not installed.
pysal 25.7 requires scikit-learn>=1.1, which is not installed.
spopt 0.7.0 requires scikit-learn>=1.4.0, which is not installed.
fastai 2.8.7 requires scikit-learn, which is not installed.
imbalanced-learn 0.14.1 requires scikit-learn<2,>=1.4.2, which is not installed.
mapclassify 2.10.0 requires scikit-learn>=1.4, which is not installed.
libpysal 4.14.1 requires scikit-learn>=1.4.0, which is not installed.
librosa 0.11.0 requires scikit-learn>=1.1.0, which is not installed.
hdbscan 0.8.42 requires scikit-learn>=1.6, which is not installed.
mlxtend 0.23.4 requires scikit-learn>=1.3.1, which is not installed.
pandas-ta 0.4.71b0 requires numpy>=2.2.6, but you have numpy 1.26.4 which is incompatible.
google-colab 1.0.0 requires pandas==2.2.2, but you have pandas 3.0.2 which is incompatible.
opencv-contrib-python 4.13.0.92 requires numpy>=2; python_version >= "3.9", but you have numpy 1.26.4 which is incompatibl
opencv-python 4.13.0.92 requires numpy>=2; python_version >= "3.9", but you have numpy 1.26.4 which is incompatible.
opencv-python-headless 4.13.0.92 requires numpy>=2; python_version >= "3.9", but you have numpy 1.26.4 which is incompatib
tobler 0.13.0 requires numpy>=2.0, but you have numpy 1.26.4 which is incompatible.
rasterio 1.5.0 requires numpy>=2, but you have numpy 1.26.4 which is incompatible.
shap 0.51.0 requires numpy>=2, but you have numpy 1.26.4 which is incompatible.
db-dtypes 1.5.1 requires pandas<3.0.0,>=1.5.3, but you have pandas 3.0.2 which is incompatible.
jax 0.7.2 requires numpy>=2.0, but you have numpy 1.26.4 which is incompatible.
gradio 5.50.0 requires pandas<3.0,>=1.0, but you have pandas 3.0.2 which is incompatible.
jaxlib 0.7.2 requires numpy>=2.0, but you have numpy 1.26.4 which is incompatible.
bqplot 0.12.45 requires pandas<3.0.0,>=1.0.0, but you have pandas 3.0.2 which is incompatible.
xarray-einstats 0.10.0 requires numpy>=2.0, but you have numpy 1.26.4 which is incompatible.
pytensor 2.38.2 requires numpy>=2.0, but you have numpy 1.26.4 which is incompatible.
Successfully installed numpy-1.26.4
---------------------------------------------------------------------------
ModuleNotFoundError                       Traceback (most recent call last)
/tmp/ipykernel_12378/445651096.py in <cell line: 0>()
      1 get_ipython().system('pip uninstall numpy scikit-learn')
      2 get_ipython().system('pip install numpy==1.26.4')
----> 3 from sklearn.model_selection import RandomizedSearchCV
      4 from sklearn.metrics import classification_report, confusion_matrix
      5 from sklearn.preprocessing import StandardScaler

ModuleNotFoundError: No module named 'sklearn'

---------------------------------------------------------------------------
NOTE: If your import is failing due to a missing package, you can
manually install dependencies using either !pip or !apt.

To view examples of installing some common dependencies, click the
"Open Examples" button below.
---------------------------------------------------------------------------
```

OPEN EXAMPLES

```
!pip install numpy scikit-learn
from sklearn.model_selection import RandomizedSearchCV
from sklearn.metrics import classification_report, confusion_matrix
from sklearn.preprocessing import StandardScaler
```

```
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (1.26.4)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-packages (1.8.0)
Requirement already satisfied: scipy>=1.10.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (1.17.1)
Requirement already satisfied: joblib>=1.3.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (1.5.3)
Requirement already satisfied: threadpoolctl>=3.2.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (3.6.0)
```

```
from sklearn.preprocessing import StandardScaler

# 1. Select only our engineered features (exclude OHLC raw prices)
```

```
feature_cols = [
    'Log_Ret', 'Z_Score_20', 'RSI', 'ROC_5', 'Dist_SMA_50',
    'BB_Width', 'ATR_Ratio', 'CMF', 'Rolling_Log_Vol_5','Day_of_Week', 'Trend'
]


# 2. Chronological Split
train_df = df[:'2020-12-31']
test_df = df['2021-01-01':]

X_train, y_train = train_df[feature_cols], train_df['Target']
X_test, y_test = test_df[feature_cols], test_df['Target']

# 3. Scaling (The right way)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train) # Learn mean/std from TRAIN only
X_test_scaled = scaler.transform(X_test)       # Apply TRAIN mean/std to TEST

print(f"Train size: {len(X_train)} | Test size: {len(X_test)}")
```

```
Train size: 2208 | Test size: 741
```

```
import xgboost as xgb
from sklearn.metrics import classification_report, confusion_matrix
from sklearn.model_selection import TimeSeriesSplit

tscv = TimeSeriesSplit(n_splits=5)
# Refined XGBoost with Regularization
model = xgb.XGBClassifier(
    n_estimators=120,
    max_depth=3,            # Shallow trees to prevent overfitting
    learning_rate=0.05,     # Slow learning is better for noisy data
    gamma=2,                # Minimum loss reduction to make a split
    subsample=0.7,
    colsample_bytree=0.8,
    eval_metric='logloss',
    use_label_encoder=False, # Suppress warning
    objective='binary:logistic' # Explicitly set for binary classification
)

model.fit(X_train_scaled, y_train)

print("XGBoost model training complete.")
```

```
XGBoost model training complete.
/usr/local/lib/python3.12/dist-packages/xgboost/training.py:200: UserWarning: [21:20:08] WARNING: /__w/xgboost/xgboost/src/l
Parameters: { "use_label_encoder" } are not used.

  bst.update(dtrain, iteration=i, fobj=obj)
```

```
from scipy.stats import uniform, randint

# Define the parameter distribution for RandomizedSearchCV
param_dist = {
    'n_estimators': randint(100, 200),  # Number of boosting rounds
    'max_depth': randint(3,6),      # Maximum tree depth
    'learning_rate': uniform(0.01, 0.05), # Step size shrinkage to prevent overfitting
    'gamma': uniform(0, 2),         # Minimum loss reduction required to make a further partition
    'subsample': uniform(0.7, 0.3),   # Subsample ratio of the training instance
    'colsample_bytree': uniform(0.6, 0.4), # Subsample ratio of columns when constructing each tree
    'lambda': uniform(1, 7),          # L2 regularization term on weights
    'alpha': uniform(0.1, 1)            # L1 regularization term on weights
}
```

```
# Initialize RandomizedSearchCV
random_search = RandomizedSearchCV(
    estimator=model,
    param_distributions=param_dist,
    n_iter=50,
    scoring='precision',
    cv=tscv,
    verbose=1,
    random_state=42,
    n_jobs=-1
)

# Fit RandomizedSearchCV to the training data
random_search.fit(X_train_scaled, y_train)
```

```
print("RandomizedSearchCV complete.")
print(f"Best parameters found: {random_search.best_params_}")
```

```
Fitting 5 folds for each of 50 candidates, totalling 250 fits
RandomizedSearchCV complete.
Best parameters found: {'alpha': 0.5497541333697656, 'colsample_bytree': 0.7580600944007257, 'gamma': 1.8533177315875884, 'l
/usr/local/lib/python3.12/dist-packages/xgboost/training.py:200: UserWarning: [21:20:55] WARNING: /__w/xgboost/xgboost/src/l
Parameters: { "use_label_encoder" } are not used.

  bst.update(dtrain, iteration=i, fobj=obj)
```

```
model = random_search.best_estimator_

# Retrain the model with the best parameters (optional, as best_estimator_ is already fitted)
# If you want to train on the entire training set with the best params, you can refit it like this:
model.fit(X_train_scaled, y_train)

print("XGBoost model retrained with best parameters.")
```

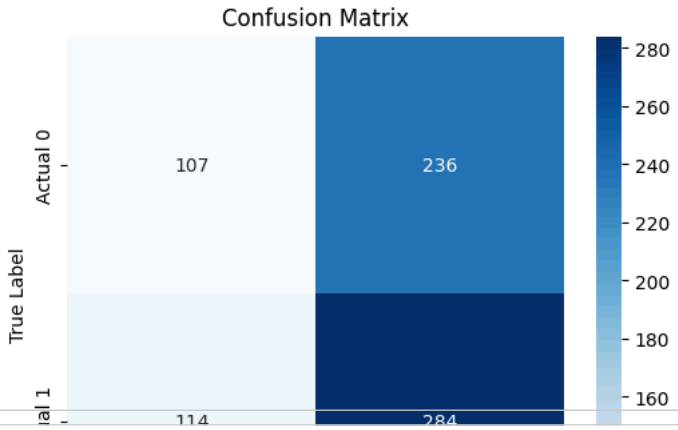
```
XGBoost model retrained with best parameters.
/usr/local/lib/python3.12/dist-packages/xgboost/training.py:200: UserWarning: [21:21:11] WARNING: /__w/xgboost/xgboost/src/l
Parameters: { "use_label_encoder" } are not used.

  bst.update(dtrain, iteration=i, fobj=obj)
```

```
y_probs = model.predict_proba(X_test_scaled)[:, 1]

# Higher threshold = Higher quality trades
threshold = 0.51
y_pred_high_conf = (y_probs >= threshold).astype(int)

print("Classification Report (High Confidence):")
print(classification_report(y_test, y_pred_high_conf))

print("\nConfusion Matrix (Tuned Model):")
print(confusion_matrix(y_test, y_pred_high_conf))
```

```
Classification Report (High Confidence):
              precision    recall  f1-score   support

           0       0.48      0.31      0.38       343
           1       0.55      0.71      0.62       398

    accuracy                           0.53       741
   macro avg       0.52      0.51      0.50       741
weighted avg       0.52      0.53      0.51       741


Confusion Matrix (Tuned Model):
[[107 236]
 [114 284]]
```

Now that the model is trained, let's make predictions on the test set and evaluate its performance.

```
import seaborn as sns
import matplotlib.pyplot as plt

# Calculate the confusion matrix
cm = confusion_matrix(y_test, y_pred_high_conf)

# Create a heatmap for better visualization
plt.figure(figsize=(6, 5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Predicted 0', 'Predicted 1'],
            yticklabels=['Actual 0', 'Actual 1'])
plt.title('Confusion Matrix')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.show()
```




```
capital = 1_000_000
capital_history = []

i = 0
indices = X_test.index

while i < len(indices) - 5:

    prob = y_probs[i]

    if prob > 0.55:
        signal = 1
    elif prob < 0.45:
        signal = 0
    else:
        i += 1
        continue

    entry_idx = indices[i+1]
    exit_idx = indices[i+5]

    entry_price = df.loc[entry_idx, 'Open']
    exit_price = df.loc[exit_idx, 'Close']

    if signal == 1:
        trade_return = (exit_price / entry_price) - 1
    else:
        trade_return = (entry_price / exit_price) - 1

    capital *= (1 + trade_return)
    capital_history.append(capital)

    i += 5
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
/tmp/ipykernel_1361/2855625453.py in <cell line: 0>()
      3
      4 i = 0
----> 5 indices = X_test.index
      6
      7 # Loop through the test set predictions, ensuring enough data for a 5-day hold

NameError: name 'X_test' is not defined
```

```
capital_series = pd.Series(capital_history)

returns = capital_series.pct_change().dropna()

sharpe = np.sqrt(252/5) * returns.mean() / returns.std()

print("Sharpe Ratio:", sharpe)
```

```
Sharpe Ratio: 0.7227929063065217
```